Performance Tuning and City Mode

How to make Immersive Engineering 1.12.2 cheap on the server tick, and what each option costs you.

The headline

You do not need to configure anything. The expensive part of the wire network has been removed in code. On a profiled world, roughly **half** of everything Immersive Engineering did was a single per-tick broadcast that existed only to feed wire-shock damage — running whether or not anything was near a wire, and running even when wire damage was switched off. That figure is now computed on demand, when an entity actually touches a wire. Nothing about gameplay changes, and no setting is involved.

Getting there took a wrong turn worth recording. The first fix was to skip the broadcast when `enableWireDamage=false`, which measured a ~60% cut — better than city mode, the feature built specifically to make the wire network cheap. That made it obvious the cost was never the physics, and the right move was to stop broadcasting entirely rather than make anyone choose.

Recommended configurations

Goal	Set	Result
Recommended — the defaults	enableWireDamage = true cityMode = false	Full realistic grid, wire damage working, and the cost that used to make people turn it off is gone for everyone.
No wire shock damage	enableWireDamage = false	A gameplay choice now, not a performance one. It used to be the biggest performance setting in the mod; it no longer is.
City / roleplay pack	cityMode = true	For the gameplay — lossless voltage-agnostic wires, no burnout — not for speed. What is left of its performance case is ~1.6% of active CPU.
Leave alone	validateConnections = false pump_placeCobble = true	Both already default. The first slows world load; the second stops flowing-fluid updates propagating.
Client FPS only	increasedRenderboxes = false disableFancyTESR = true	No effect on TPS. For low-end GPUs.

What city mode is

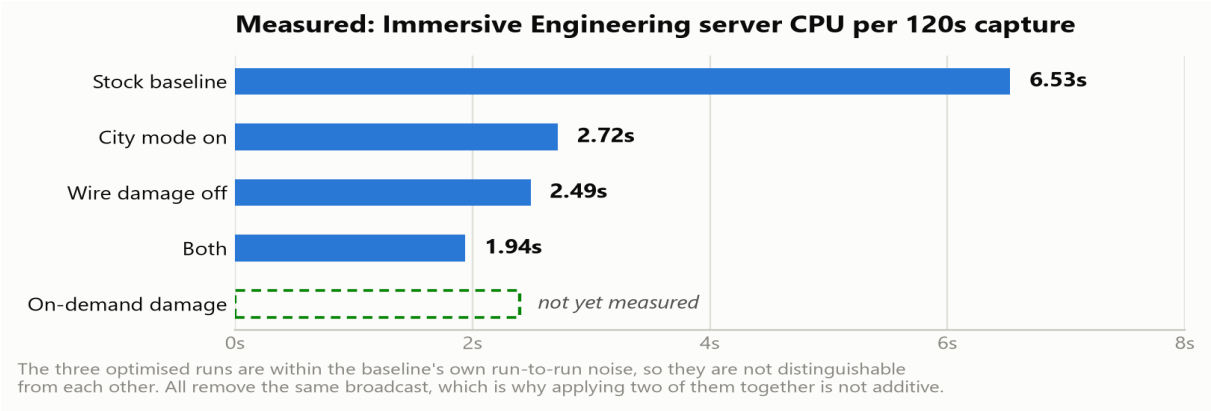
City mode trades simulation detail for server tick time. It is aimed at city and roleplay packs where the mod's machinery is set dressing rather than an engineering puzzle: you keep the entire look of the build and give up the physics behind it. It covers five subsystems.

Subsystem	What is simplified
Wires	One lossless push per connector instead of loss, distance weighting, proportional split, a double simulate/real pass and a network-wide broadcast.
Floodlights	Beams re-traced only when the light switches or a neighbour changes, never on a timer; light-block count capped per lamp.

Subsystem	What is simplified
Generators	Fuel becomes cosmetic — a presence check and a token sip instead of a per-tick burn rate and tank drain.
Machines	Idle multiblocks stop re-scanning the recipe list every tick; the scan interval widens.
Virtual grid	Segments stop accounting for flux and switch to presence: energized or not, with Service Units delivering freely and Feed Units taking only a token sip.

It is **off by default**, and off means byte-identical to stock. `cityMode` is the master switch; `cityModeWires`, `cityModeFloodlights`, `cityModeGenerators`, `cityModeMachines` and `cityModeVirtualGrid` each default to on, so the master alone enables everything while any one subsystem can be declined. Switching the master off is always sufficient to restore stock behaviour, and nothing here touches saved data.

Measured results



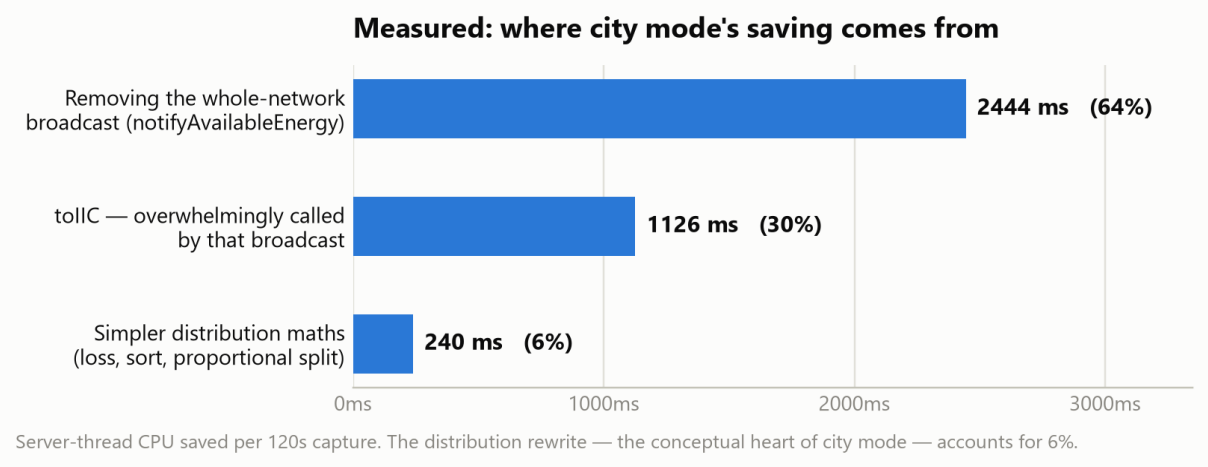
Measured, not modelled. Five 120-second spark captures of the Server thread on a local world, flying along power lines — the worst case for the route cache, since chunk streaming keeps invalidating it. Library time is attributed to the calling mod and idle (the server thread sleeps 84–92% of the time on an unsaturated world) is excluded.

Read the chart carefully. The two baseline runs differ by 30% from each other, which puts this methodology's noise floor around ±13%. The three optimised runs span a smaller range than that, so they are *not* distinguishable from one another — an earlier version of this document claimed wire-damage-off beat city mode on the strength of that gap, and that claim was wrong. What the runs do show is that combining two of them is not additive, which is exactly what you would see if a single shared bottleneck dominated.

The physics is nearly free. The wire-damage-off run keeps the entire realistic distribution — per-wire loss, the TreeMap sort by loss rate, proportional splitting, the double simulate/real pass, the burnout ledger — and transferEnergy costs 1212 ms against city mode's stripped-down cityModeTransfer at 996 ms. That 216 ms is the price of everything city mode removes from power behaviour: about 1.6% of active CPU.

The floodlight, machine and generator subsystems did not register in these captures — TileEntityFloodlight.update came in at 0.04%, effectively absent. That neither vindicates nor refutes them: the profiled area simply had no lit floodlights and no idle machines holding unusable input. Their value depends entirely on what is built and loaded, so measure them where they exist, toggling the sub-flags individually. The floodlight work was predicted to rival the wire saving in a lit city; that prediction remains unmeasured.

Where the saving actually comes from



This was a surprise, and it overturned the estimate this document previously carried. City mode's conceptual heart — replacing the loss maths, the TreeMap sort by loss rate, the proportional split and the double simulate/real pass with a single lossless push — accounts for about 6% of the saving. Essentially all of it is the removal of one whole-network broadcast per connector per tick.

That has a consequence worth acting on. The broadcast exists solely to feed wire-shock damage, so most of this performance is available *without* giving up loss, voltage tiers or wire burnout. The broadcast now respects enableWireDamage — it previously ran every tick even with the feature switched off — and computing damage lazily on entity-wire collision would give the realistic grid most of city mode's speedup.

One thing did *not* improve: getIndirectEnergyConnections was unchanged despite being called once per tick instead of three times, so its cost is cache misses being re-flooded rather than call count. It is now the largest remaining Immersive Engineering cost.

What changes, in gameplay terms

	Normal	City mode
Wire loss	2.5–5% per 16 blocks, worse when lightly loaded	none
Voltage tiers	throttle to the weakest wire on the path	cosmetic only
Supply < demand	proportional to demand, nearest first	greedy, arbitrary order
Wire burnout	wire destroyed above its rate	cannot happen
Wire shock damage	sourced from the whole network	sourced from the local connector
Breaker switches	cut the network	unchanged
Energy meters	read loss-attenuated throughput	read full throughput
Floodlight beams	re-traced every 512 ticks	only on switch / neighbour change
Floodlight light count	uncapped	capped at 64 per lamp
Generator fuel burn	per-tick rate from the fluid	1 mB every 20 ticks, cosmetic

	Normal	City mode
Generator load gate	only runs under load	**unchanged — deliberately kept**
Idle machine recipe scan	every tick	every 32 ticks
Machine speed / power needs	—	unchanged

Three consequences worth knowing

Floodlights stop noticing distant obstructions

A floodlight is usually the most expensive block in a city build. Every 512 ticks each one re-traces thirteen beams, queues a light block roughly every third block along each, and recalculates block lighting — and every light it places is an individually ticking tile entity, uncapped, so one unobstructed lamp can own well over a hundred. City mode rebuilds only when the light switches or a neighbouring block changes. The cost is that a wall built across a beam further out is not noticed until something else triggers a rebuild, leaving the lights beyond it floating until the lamp is toggled.

Wire burnout is disabled

Overload destruction works by the normal transfer path recording per-connection throughput into a ledger that a world-tick handler then acts on. That path is the ledger's only writer, so in city mode it stays empty and no wire can ever burn out. Combined with the fact that city mode never clamps to the cable's rate, a copper wire will carry a full HV connector's 4096 FE/t forever with no consequence. That is intended for a city pack, but it is a rules change rather than an optimisation.

Wire shock damage becomes local

Damage is computed from a per-tick list of energy sources on each connectable, which the skipped network broadcast used to populate. In city mode that list holds only the connector's own energy, so damage is generally lower and a wire span running between two relays will not shock at all, because relays never hold energy. Set `enableWireDamage = false` to turn it off properly.

Does a diesel generator still need fuel?

Yes. A diesel generator gates on two conditions: something must actually accept power, and there must be fuel in the tank. With no consumers, connector buffers saturate, the generator's simulated insert returns zero, the fan spins down and no fuel burns — in both modes. What city mode makes cosmetic is the *rate*: instead of deriving a per-tick burn rate from the fluid and draining every tick, it sips 1 mB every 20 ticks, so a full tank lasts about six and a half hours of runtime.

The load gate is kept deliberately. Only running when something wants power is a *performance* feature, not a realism one — it is what makes an idle generator free. Removing it would make city mode slower, not faster, because generators would push energy at saturated connectors every tick only for it to be discarded.

City mode also raises the yield per unit of fuel, because the 4096 FE/t leaving the generator arrives undiminished instead of being attenuated by every wire segment on the way. More useful power per bucket of biodiesel, but never power from nothing — only what receivers actually accepted is deducted from the connector.

Block-by-block summary

Block	Role	Affected by city mode
Diesel Generator	4096 FE/t, burns fuel only under load	**Yes** — fuel burn becomes a token sip; load gate kept
Thermoelectric / Dynamo	push to adjacent blocks only	No — already cached or event-driven
Windmill / Water Wheel	drive the dynamo, produce no FE	No — already cached and throttled
Floodlight	13 beams of ticking light blocks	**Yes** — no timed re-scan, 64-light cap
Multiblock machines	Arc Furnace, Squeezer, Fermenter, Mixer, Refinery	**Yes** — idle recipe scan throttled to 32 ticks
Capacitor LV/MV/HV	buffer; 256 / 1024 / 4096 FE/t	No
Creative Capacitor	infinite source, all six sides	No — works as an infinite source in both modes
Lightning Rod	16,000,000 FE on a strike	No
Connector LV/MV/HV	the only blocks moving energy over wires	**Yes** — push path replaced; its own rate caps still apply
Relay LV/MV/HV	routing waypoint, never holds energy	No — tick body never runs either way
Grid Feed / Service Unit	wireless segment endpoints; never tick individually	**Yes** — pool accounting replaced by a presence check
Transformer	tier adapter; holds no energy	**Yes** — tier throttling disappears
Breaker Switch	interrupts the network	No — still cuts
Energy Meter	passive throughput accumulator	Works, and reads the full unattenuated amount
Machine power intake	receive by push into their own buffer	No — their own intake caps still apply
Direct generator → machine	no wires involved	No — entirely unaffected

Full technical detail, including the annotated transfer routine, wire-type tables and the testing checklist, is in `docs/CITY_MODE_AND_PERF.md`.